modelo

August 10, 2025

[1]:
```
# Importação das bibliotecas necessárias para manipulação de dados, modelagem e␣
↪avaliação
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
↪f1_score, confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import shap
```

[2]:
```
# Leitura dos dados da planilha Excel
df = pd.read_excel('DADOS/VisaoGeralArinos.xlsx', sheet_name='Planilha1')
```

[3]:
```
# Conversão das colunas para valores numéricos, tratando valores '(Em branco)'␣
↪como zero
df['Concluintes'] = pd.to_numeric(
    df['Concluintes'].replace('(Em branco)', 0),
    errors='coerce'
).fillna(0)

df['Matrículas'] = pd.to_numeric(
    df['Matriculas'].replace('(Em branco)', 0),
    errors='coerce'
).fillna(0)

df['Ingressantes'] = pd.to_numeric(
    df['Ingressantes'].replace('(Em branco)', 0),
```

```
    errors='coerce'
).fillna(0)
```

```
/tmp/ipykernel_2923/1710126678.py:3: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to
the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  df['Concluintes'].replace('(Em branco)', 0),
/tmp/ipykernel_2923/1710126678.py:13: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to
the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  df['Ingressantes'].replace('(Em branco)', 0),
```

[4]:
```
# Visualização inicial dos dados
df.head()
```

[4]:
```
  Instituicao ClassificacaoRacial RendaFamiliar      Sexo FaixaEtaria  \
0       IFNMG             Amarela    0<RFP<=0,5  Feminino     20 a 24
1       IFNMG             Amarela    0<RFP<=0,5  Feminino     20 a 24
2       IFNMG             Amarela    0<RFP<=0,5  Feminino     20 a 24
3       IFNMG             Amarela    0<RFP<=0,5  Feminino     25 a 29
4       IFNMG             Amarela    0<RFP<=0,5  Feminino     25 a 29

       FonteFinanciamento   Ano  Matriculas  Ingressantes  Concluintes  \
0  Recursos Orçamentários  2017           1             1            0
1  Recursos Orçamentários  2019           1             0            0
2  Sem Programa Associado  2018           1             0            0
3  Sem Programa Associado  2020           1             0            0
4  Sem Programa Associado  2021           1             0            0

   Matrículas
0           1
1           1
2           1
3           1
4           1
```

[5]:
```
# Agrupamento dos dados para obter totais por grupo de características
matriculas_atuais = df[df['Ano'] == df['Ano'].max()].groupby(
    ['ClassificacaoRacial', 'RendaFamiliar', 'Sexo', 'FaixaEtaria',␣
 ↪'FonteFinanciamento']
)['Matriculas'].sum().reset_index()

agrupados_totais = df.groupby(
    ['ClassificacaoRacial', 'RendaFamiliar', 'Sexo', 'FaixaEtaria',␣
 ↪'FonteFinanciamento']
```

```
)[['Concluintes', 'Ingressantes']].sum().reset_index()

# Junção dos dados agrupados
resultado_final = pd.merge(agrupados_totais, matriculas_atuais, on=[
    'ClassificacaoRacial', 'RendaFamiliar', 'Sexo', 'FaixaEtaria',␣
 ↪'FonteFinanciamento'
], how='left').fillna(0)
```

[6]:
```
# Visualização do resultado final do agrupamento
resultado_final
```

[6]:
```
    ClassificacaoRacial  RendaFamiliar       Sexo FaixaEtaria  \
0               Amarela     0,5<RFP<=1  Masculino     20 a 24
1               Amarela     0,5<RFP<=1  Masculino     25 a 29
2               Amarela     0<RFP<=0,5   Feminino     20 a 24
3               Amarela     0<RFP<=0,5   Feminino     20 a 24
4               Amarela     0<RFP<=0,5   Feminino     25 a 29
..                  …            …        …         …
194               Preta  Não declarada  Masculino     15 a 19
195               Preta  Não declarada  Masculino     20 a 24
196               Preta        RFP>3,5  Masculino     30 a 34
197               Preta        RFP>3,5  Masculino     30 a 34
198               Preta        RFP>3,5  Masculino     35 a 39

         FonteFinanciamento  Concluintes  Ingressantes  Matriculas
0    Sem Programa Associado            0             1         0.0
1    Sem Programa Associado            0             0         1.0
2    Recursos Orçamentários            0             1         0.0
3    Sem Programa Associado            0             0         0.0
4    Sem Programa Associado            1             0         0.0
..                       …          …            …          …
194  Sem Programa Associado            0             1         0.0
195  Sem Programa Associado            0             2         1.0
196  Recursos Orçamentários            0             0         0.0
197  Sem Programa Associado            0             1         0.0
198  Sem Programa Associado            0             0         1.0

[199 rows x 8 columns]
```

[7]:
```
# Criação do DataFrame de alunos, simulando a situação de cada um (Evadido,␣
 ↪Matriculado, Concluído)
df_alunos = pd.DataFrame(columns=[
    'ClassificacaoRacial', 'RendaFamiliar', 'Sexo', 'FaixaEtaria',
    'FonteFinanciamento', 'Situacao'
])

for row in resultado_final.itertuples():
```

```
    # Adiciona alunos evadidos
    for i in range(0, int(row.Ingressantes) - (int(row.Concluintes) + int(row.
↪Matriculas))):
        df_situacao = pd.DataFrame({
            'ClassificacaoRacial': row.ClassificacaoRacial,
            'RendaFamiliar': row.RendaFamiliar,
            'Sexo': row.Sexo,
            'FaixaEtaria': row.FaixaEtaria,
            'FonteFinanciamento': row.FonteFinanciamento,
            'Situacao': 'Evadido'
        }, index=[0])

        df_alunos = pd.concat([df_alunos, df_situacao], ignore_index=True)

    # Adiciona alunos matriculados
    for i in range(0, int(row.Matriculas)):
        df_situacao = pd.DataFrame({
            'ClassificacaoRacial': row.ClassificacaoRacial,
            'RendaFamiliar': row.RendaFamiliar,
            'Sexo': row.Sexo,
            'FaixaEtaria': row.FaixaEtaria,
            'FonteFinanciamento': row.FonteFinanciamento,
            'Situacao': 'Matriculado'
        }, index=[0])

        df_alunos = pd.concat([df_alunos, df_situacao], ignore_index=True)

    # Adiciona alunos concluídos
    for i in range(0, int(row.Concluintes)):
        df_situacao = pd.DataFrame({
            'ClassificacaoRacial': row.ClassificacaoRacial,
            'RendaFamiliar': row.RendaFamiliar,
            'Sexo': row.Sexo,
            'FaixaEtaria': row.FaixaEtaria,
            'FonteFinanciamento': row.FonteFinanciamento,
            'Situacao': 'Concluído'
        }, index=[0])

        df_alunos = pd.concat([df_alunos, df_situacao], ignore_index=True)

df_alunos.head()
```

[7]:

```
  ClassificacaoRacial RendaFamiliar       Sexo FaixaEtaria  \
0             Amarela    0,5<RFP<=1  Masculino     20 a 24
1             Amarela    0,5<RFP<=1  Masculino     25 a 29
2             Amarela    0<RFP<=0,5   Feminino     20 a 24
3             Amarela    0<RFP<=0,5   Feminino     25 a 29
```

```
4             Amarela    0<RFP<=0,5  Masculino     25 a 29

       FonteFinanciamento     Situacao
0  Sem Programa Associado      Evadido
1  Sem Programa Associado  Matriculado
2  Recursos Orçamentários      Evadido
3  Sem Programa Associado    Concluído
4  Recursos Orçamentários      Evadido
```

[8]:
```
# Visualização do DataFrame de alunos
df_alunos
```

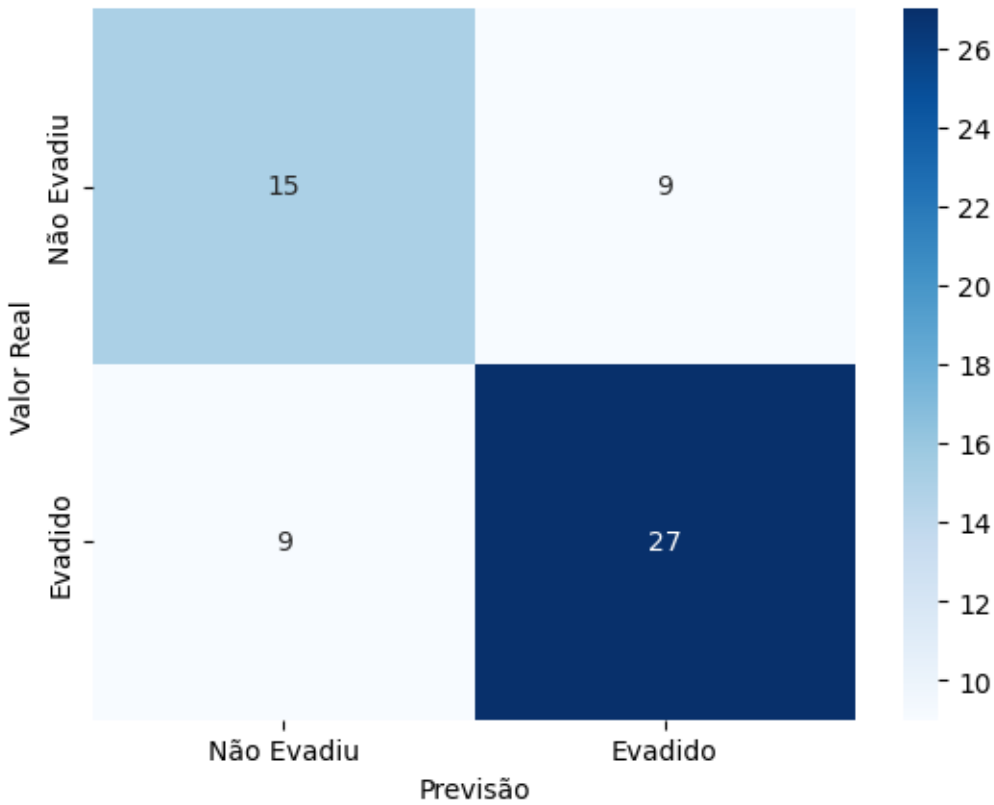
[8]:
```
    ClassificacaoRacial  RendaFamiliar       Sexo FaixaEtaria  \
0               Amarela     0,5<RFP<=1  Masculino     20 a 24
1               Amarela     0,5<RFP<=1  Masculino     25 a 29
2               Amarela     0<RFP<=0,5   Feminino     20 a 24
3               Amarela     0<RFP<=0,5   Feminino     25 a 29
4               Amarela     0<RFP<=0,5  Masculino     25 a 29
..                  …            …        …         …
291               Preta  Não declarada  Masculino     15 a 19
292               Preta  Não declarada  Masculino     20 a 24
293               Preta  Não declarada  Masculino     20 a 24
294               Preta        RFP>3,5  Masculino     30 a 34
295               Preta        RFP>3,5  Masculino     35 a 39

         FonteFinanciamento     Situacao
0    Sem Programa Associado      Evadido
1    Sem Programa Associado  Matriculado
2    Recursos Orçamentários      Evadido
3    Sem Programa Associado    Concluído
4    Recursos Orçamentários      Evadido
..                      …          …
291  Sem Programa Associado      Evadido
292  Sem Programa Associado      Evadido
293  Sem Programa Associado  Matriculado
294  Sem Programa Associado      Evadido
295  Sem Programa Associado  Matriculado

[296 rows x 6 columns]
```

[9]:
```
# Mapeamento de valores categóricos para valores numéricos (renda e faixa␣
↪etária)
mapa_renda = {
    '0,5<RPF<=1': 1125,
    '0<RPF<=0,5': 375,
    '1,5<RFP<=2,5': 3000,
    '1<RFP<=1,5': 1875,
```

```
    '2,5<RFP<=3,5': 4500,
    'Não declarada': 0,
    'RFP>3,5': 5250
}

mapa_faixa = {
    '18 a 24': 21,
    '25 a 34': 29.5,
    '35 a 44': 39.5,
    '45 a 54': 49.5,
    '55 a 64': 59.5,
    '65 ou mais': 70
}

# Aplicação dos mapeamentos
df_alunos['RendaFamiliarNum'] = df_alunos['RendaFamiliar'].map(mapa_renda).
 ↪fillna(0)
df_alunos['FaixaEtariaNum'] = df_alunos['FaixaEtaria'].map(mapa_faixa).fillna(0)

# Definição das colunas para o modelo
colunas_modelo = [
    'ClassificacaoRacial', 'Sexo', 'FonteFinanciamento', 'RendaFamiliarNum',␣
 ↪'FaixaEtariaNum'
]

colunas_categoricas_novas = ['ClassificacaoRacial', 'Sexo',␣
 ↪'FonteFinanciamento']

# Pré-processador para transformar variáveis categóricas e normalizar as␣
 ↪numéricas
preprocessador = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),␣
 ↪colunas_categoricas_novas),
        ('num', StandardScaler(), ['RendaFamiliarNum', 'FaixaEtariaNum'])
    ]
)

# Separação das features e do alvo
X = df_alunos[colunas_modelo]
y = (df_alunos['Situacao'] == 'Evadido').astype(int)

# Transformação dos dados
X_transformado = preprocessador.fit_transform(X)
```

[10]:
```python
# Separação dos dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42, stratify=y)

# Pipeline com regressão logística
pipeline_modelo = Pipeline(steps=[
    ('preprocessador', preprocessador),
    ('classificador', LogisticRegression(max_iter=1000))
])

pipeline_modelo.fit(X_train, y_train)

# Avaliação do modelo
y_pred = pipeline_modelo.predict(X_test)

# Métricas
print("Acurácia:", accuracy_score(y_test, y_pred))
print("Precisão:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred))

# Relatório detalhado
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))

# Matriz de confusão visual
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Não Evadiu',␣
↪'Evadido'], yticklabels=['Não Evadiu', 'Evadido'])
plt.ylabel('Valor Real')
plt.xlabel('Previsão')
plt.show()
```

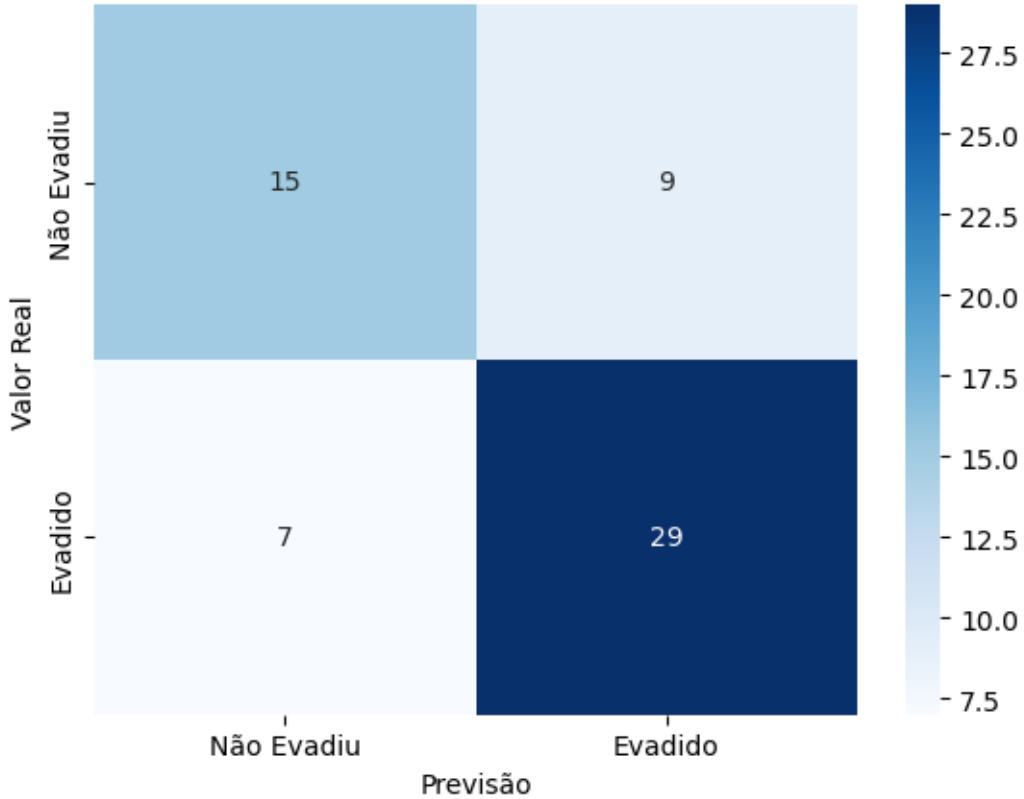
```
Acurácia: 0.7
Precisão: 0.75
Recall: 0.75
F1-Score: 0.75

Relatório de Classificação:
               precision    recall  f1-score   support

           0       0.62      0.62      0.62        24
           1       0.75      0.75      0.75        36

    accuracy                           0.70        60
   macro avg       0.69      0.69      0.69        60
weighted avg       0.70      0.70      0.70        60
```



[11]:
```
# Pipeline com Random Forest
pipeline_rf = Pipeline(steps=[
    ('preprocessador', preprocessador),
    ('classificador', RandomForestClassifier(n_estimators=100, random_state=42))
])

pipeline_rf.fit(X_train, y_train)

# Avaliação do modelo
y_pred = pipeline_rf.predict(X_test)

# Métricas
print("Acurácia:", accuracy_score(y_test, y_pred))
print("Precisão:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred))

# Relatório detalhado
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))

# Matriz de confusão visual
```

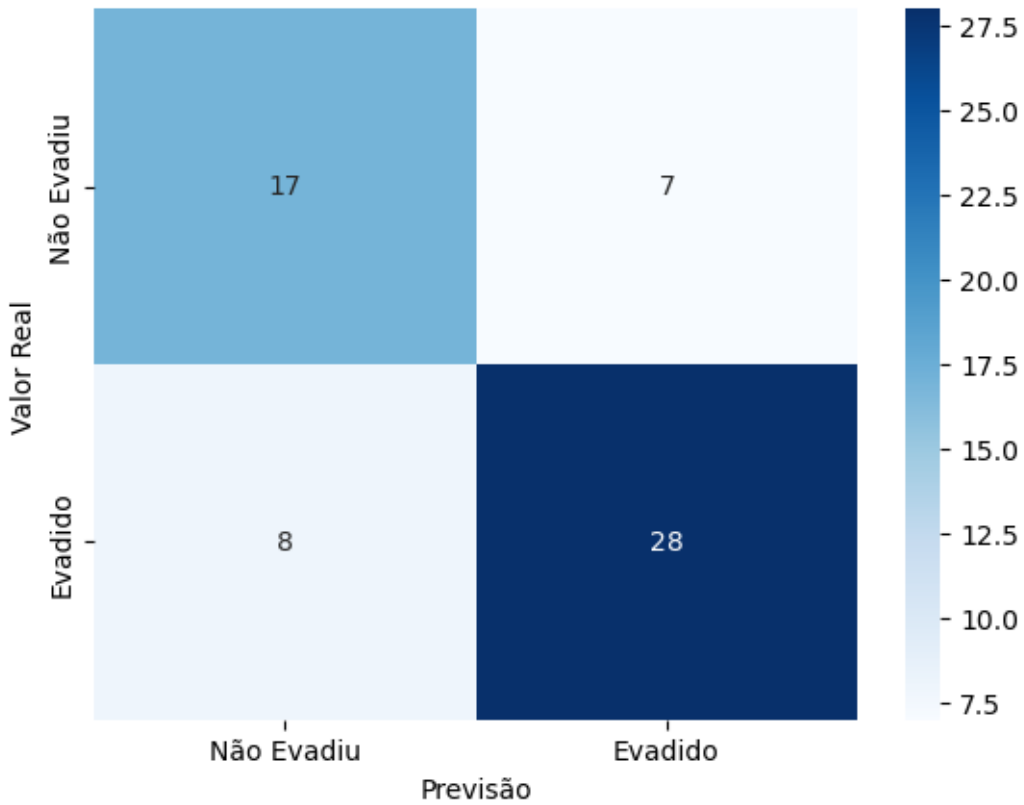
```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Não Evadiu',␣
↪'Evadido'], yticklabels=['Não Evadiu', 'Evadido'])
plt.ylabel('Valor Real')
plt.xlabel('Previsão')
plt.show()
```

```
Acurácia: 0.7333333333333333
Precisão: 0.7631578947368421
Recall: 0.8055555555555556
F1-Score: 0.7837837837837838

Relatório de Classificação:
              precision    recall  f1-score   support

           0       0.68      0.62      0.65        24
           1       0.76      0.81      0.78        36

    accuracy                           0.73        60
   macro avg       0.72      0.72      0.72        60
weighted avg       0.73      0.73      0.73        60
```



[12]:
```python
# Pipeline com Gradient Boosting
pipeline_gb = Pipeline(steps=[
    ('preprocessador', preprocessador),
    ('classificador', GradientBoostingClassifier(n_estimators=100,␣
 ↪random_state=42))
])

pipeline_gb.fit(X_train, y_train)

# Avaliação do modelo
y_pred = pipeline_gb.predict(X_test)

# Métricas
print("Acurácia:", accuracy_score(y_test, y_pred))
print("Precisão:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred))

# Relatório detalhado
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))

# Matriz de confusão visual
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Não Evadiu',␣
 ↪'Evadido'], yticklabels=['Não Evadiu', 'Evadido'])
plt.ylabel('Valor Real')
plt.xlabel('Previsão')
plt.show()
```

```
Acurácia: 0.75
Precisão: 0.8
Recall: 0.7777777777777778
F1-Score: 0.7887323943661971

Relatório de Classificação:
               precision    recall  f1-score   support

           0       0.68      0.71      0.69        24
           1       0.80      0.78      0.79        36

    accuracy                           0.75        60
   macro avg       0.74      0.74      0.74        60
weighted avg       0.75      0.75      0.75        60
```

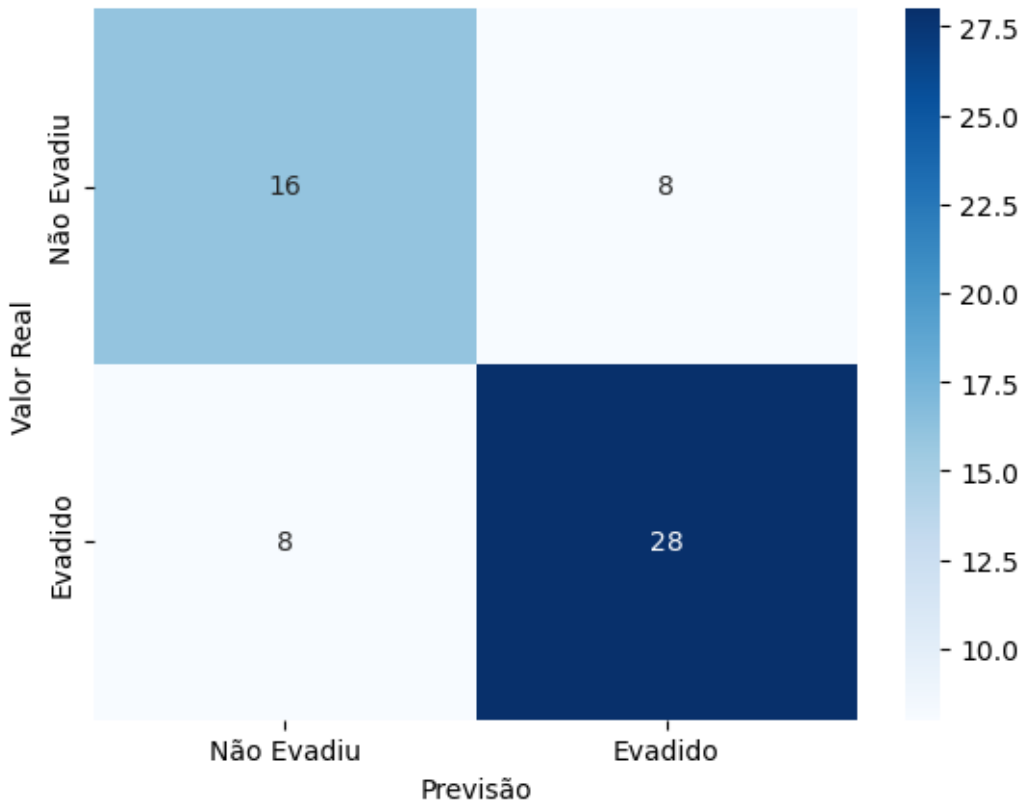


[13]:
```
# Pipeline com XGBoost
pipeline_xgb = Pipeline(steps=[
    ('preprocessador', preprocessador),
    ('classificador', XGBClassifier(n_estimators=100, random_state=42,␣
 ↪use_label_encoder=False, eval_metric='logloss'))
])

pipeline_xgb.fit(X_train, y_train)

# Avaliação do modelo
y_pred = pipeline_xgb.predict(X_test)

# Métricas
print("Acurácia:", accuracy_score(y_test, y_pred))
print("Precisão:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred))

# Relatório detalhado
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))
```

```python
# Matriz de confusão visual
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Não Evadiu',␣
↪'Evadido'], yticklabels=['Não Evadiu', 'Evadido'])
plt.ylabel('Valor Real')
plt.xlabel('Previsão')
plt.show()
```

```
/home/ogarit/tcc/.pixi/envs/default/lib/python3.13/site-
packages/xgboost/training.py:183: UserWarning: [23:00:26] WARNING:
/home/conda/feedstock_root/build_artifacts/xgboost-
split_1754002079811/work/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

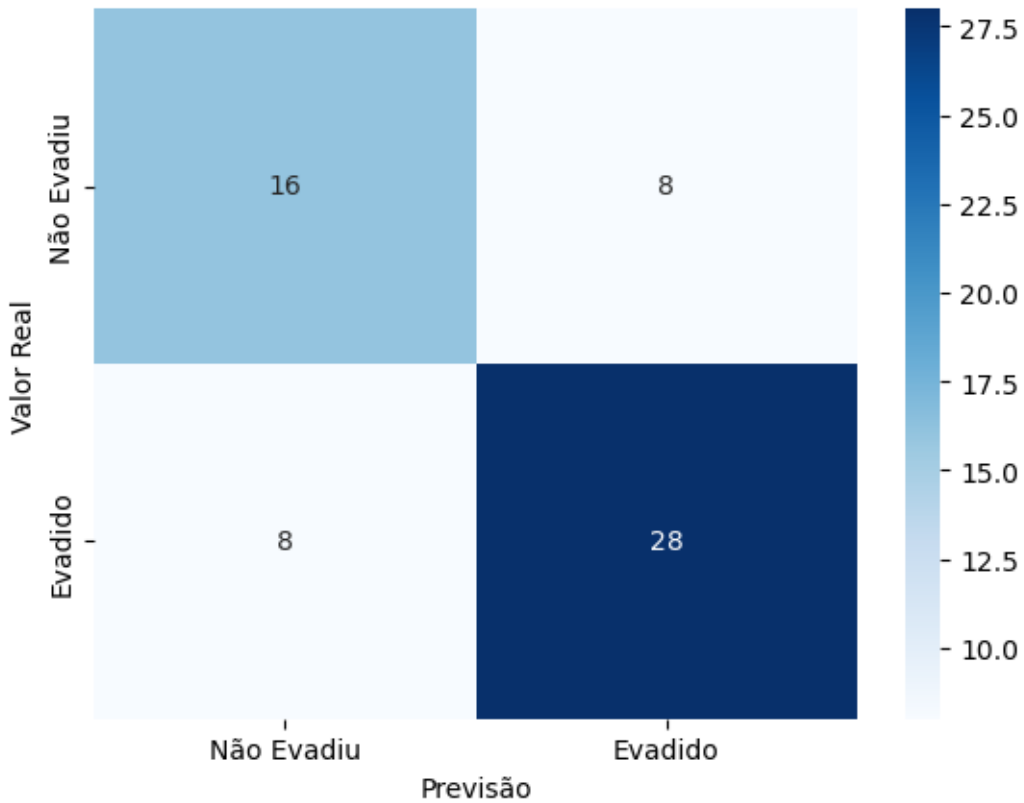
```
Acurácia: 0.7333333333333333
Precisão: 0.7777777777777778
Recall: 0.7777777777777778
F1-Score: 0.7777777777777778

Relatório de Classificação:
               precision    recall  f1-score   support

           0       0.67      0.67      0.67        24
           1       0.78      0.78      0.78        36

    accuracy                           0.73        60
   macro avg       0.72      0.72      0.72        60
weighted avg       0.73      0.73      0.73        60
```



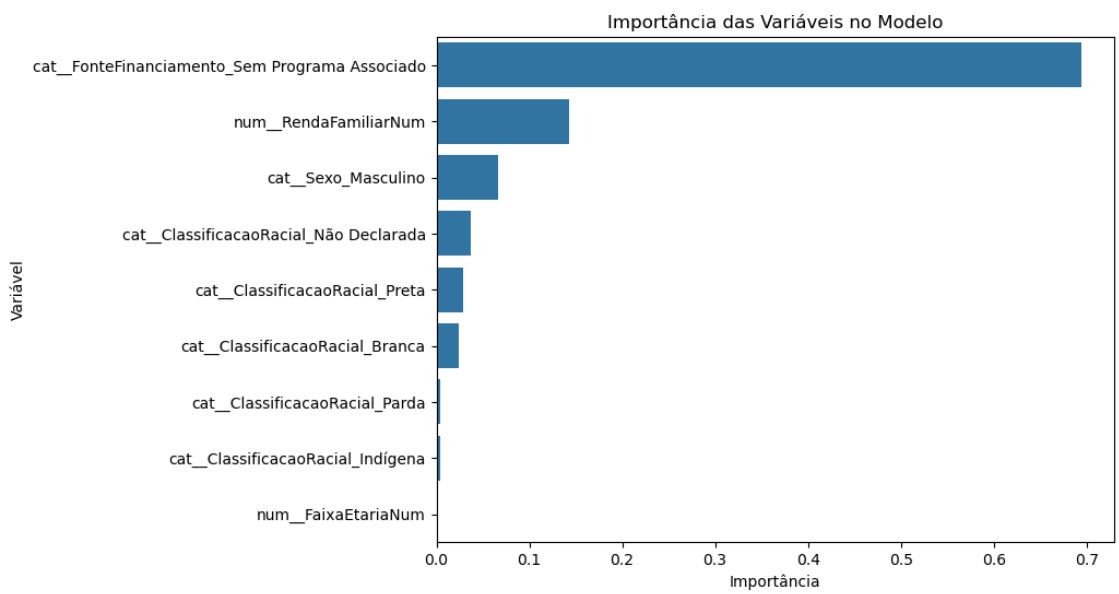
[14]:
```
# Pipeline com Decision Tree
pipeline_dt = Pipeline(steps=[
    ('preprocessador', preprocessador),
    ('classificador', DecisionTreeClassifier(random_state=42))
])

pipeline_dt.fit(X_train, y_train)

# Avaliação do modelo
y_pred = pipeline_xgb.predict(X_test)

# Métricas
print("Acurácia:", accuracy_score(y_test, y_pred))
print("Precisão:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred))

# Relatório detalhado
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))

# Matriz de confusão visual
```

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Não Evadiu',␣
↪'Evadido'], yticklabels=['Não Evadiu', 'Evadido'])
plt.ylabel('Valor Real')
plt.xlabel('Previsão')
plt.show()
```

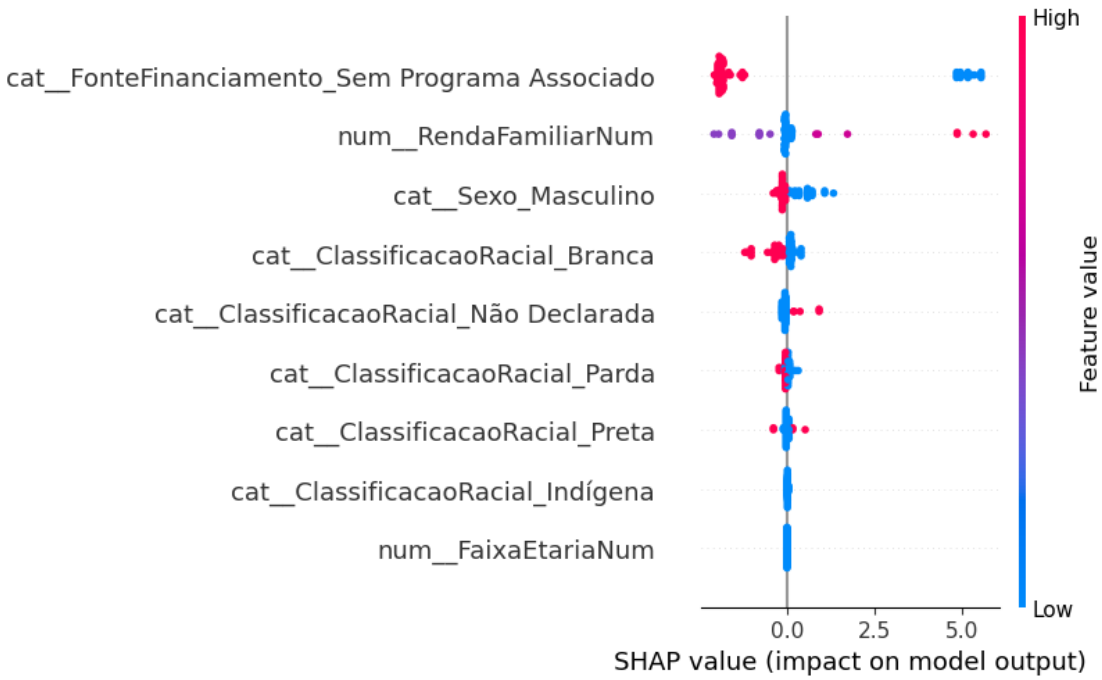
```
Acurácia: 0.7333333333333333
Precisão: 0.7777777777777778
Recall: 0.7777777777777778
F1-Score: 0.7777777777777778

Relatório de Classificação:
              precision    recall  f1-score   support

           0       0.67      0.67      0.67        24
           1       0.78      0.78      0.78        36

    accuracy                           0.73        60
   macro avg       0.72      0.72      0.72        60
weighted avg       0.73      0.73      0.73        60
```



[15]:
```
# Nomes das variáveis após o pré-processamento
nomes_variaveis = pipeline_gb.named_steps['preprocessador'].
 ↪get_feature_names_out()

# Importância das variáveis
importancias = pipeline_gb.named_steps['classificador'].feature_importances_
df_importancia = pd.DataFrame({'Variável': nomes_variaveis, 'Importância':␣
 ↪importancias})
df_importancia = df_importancia.sort_values(by='Importância', ascending=False)

# Plot
plt.figure(figsize=(8, 6))
sns.barplot(x='Importância', y='Variável', data=df_importancia)
plt.title('Importância das Variáveis no Modelo')
plt.show()
```



[16]:
```
# Cria explicador SHAP
explainer = shap.TreeExplainer(pipeline_gb.named_steps['classificador'])

# Calcula valores SHAP
X_test_transf = pipeline_gb.named_steps['preprocessador'].transform(X_test)
shap_values = explainer.shap_values(X_test_transf)

# Resumo da importância das variáveis
shap.summary_plot(shap_values, X_test_transf, feature_names=nomes_variaveis)
```



[17]:
```
# Salvando o modelo treinado (Gradient Boosting) para uso futuro
joblib.dump(modelo_final, 'modelo_evasao.pkl')
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[17], line 2
      1 # Salvando o modelo treinado (Gradient Boosting) para uso futuro
----> 2 joblib.dump(modelo_final, 'modelo_evasao.pkl')

NameError: name 'modelo_final' is not defined
```